

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 15 – Manual do Desenvolvedor

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

1 – Introdução

1.1 Objetivo

Este documento tem como objetivo servir como **Manual do Desenvolvedor do Gestor de Territórios e Publicações (GTP)**.

Seu propósito é reunir todas as informações necessárias para que um desenvolvedor possa configurar o ambiente, compreender a arquitetura da solução, seguir os padrões estabelecidos, contribuir com novas funcionalidades e manter o sistema de forma consistente.

Este manual complementa a documentação arquitetural do projeto, oferecendo uma visão prática do processo de desenvolvimento.

1.2 Público-Alvo

Este documento destina-se a:

- Desenvolvedores Frontend;
- Desenvolvedores Backend;
- Desenvolvedores Full Stack;
- DevOps;
- Arquitetos de Software;
- Testadores;
- Novos colaboradores;
- Mantenedores do projeto.

1.3 Objetivos do Manual

Este manual possui os seguintes objetivos:

- padronizar o desenvolvimento;
- facilitar a entrada de novos desenvolvedores;
- reduzir tempo de onboarding;
- documentar convenções adotadas;

- evitar divergências entre implementações;
- facilitar manutenção futura;
- servir como guia oficial do projeto.

1.4 Visão Geral do Projeto

O **Gestor de Territórios e Publicações (GTP)** é uma plataforma web destinada ao gerenciamento das atividades relacionadas aos territórios e publicações utilizados pelas congregações.

O sistema foi projetado seguindo princípios modernos de engenharia de software, priorizando:

- arquitetura limpa;
- modularização;
- escalabilidade;
- segurança;
- facilidade de manutenção;
- alta testabilidade;
- documentação abrangente.

1.5 Tecnologias Utilizadas

Backend

- Java 21 LTS
- Spring Boot 3
- Spring Security
- Spring Data JPA
- Hibernate
- Flyway
- Maven
- PostgreSQL

Frontend

- React 19
- Vite
- React Router
- Axios
- Context API
- CSS Modules / CSS Moderno

Infraestrutura

- Docker
- Docker Compose
- GitHub Actions
- Prometheus
- Grafana
- Nginx

1.6 Estrutura do Manual

Este documento está organizado da seguinte forma:

- **Capítulo 1** – Introdução
- **Capítulo 2** – Preparação do Ambiente de Desenvolvimento
- **Capítulo 3** – Estrutura do Projeto
- **Capítulo 4** – Padrões de Desenvolvimento
- **Capítulo 5** – Fluxo de Trabalho (Git e GitHub)
- **Capítulo 6** – Execução do Projeto
- **Capítulo 7** – Debug, Logs e Solução de Problemas
- **Capítulo 8** – Testes e Qualidade
- **Capítulo 9** – Contribuição para o Projeto
- **Capítulo 10** – Boas Práticas, Checklist do Desenvolvedor e Conclusão

1.7 Convenções Utilizadas

Durante todo o projeto serão adotadas convenções padronizadas para:

- nomenclatura de arquivos;
- nomenclatura de classes;
- nomenclatura de componentes React;
- nomenclatura de APIs;
- nomenclatura de tabelas;
- organização de pacotes;
- mensagens de commit;
- estrutura de branches.

Essas convenções serão detalhadas nos capítulos seguintes.

1.8 Filosofia do Projeto

O desenvolvimento do GTP segue alguns princípios fundamentais:

- simplicidade;
- código limpo (*Clean Code*);
- arquitetura limpa (*Clean Architecture*);
- responsabilidade única (*Single Responsibility Principle*);
- baixo acoplamento;
- alta coesão;
- documentação antes da implementação;
- testes automatizados;
- melhoria contínua.

1.9 Considerações Iniciais

O Manual do Desenvolvedor representa o principal guia técnico para implementação e evolução do GTP.

Seu objetivo é garantir que todos os desenvolvedores trabalhem seguindo os mesmos padrões, reduzindo inconsistências e facilitando a manutenção da aplicação ao longo do tempo.

À medida que o projeto evoluir, este documento deverá ser revisado para refletir novas tecnologias, ferramentas e práticas adotadas pela equipe.

1.10 Conclusão do Capítulo

Este capítulo apresentou a finalidade, o escopo e a organização do Manual do Desenvolvedor, estabelecendo a base para os capítulos seguintes. Com uma visão geral do projeto, das tecnologias empregadas e dos princípios de desenvolvimento, o documento passa a servir como referência central para todos os colaboradores envolvidos na construção e manutenção do GTP.

2 – Preparação do Ambiente de Desenvolvimento

2.1 Objetivo

Este capítulo apresenta os requisitos e procedimentos necessários para configurar o ambiente de desenvolvimento do **Gestor de Territórios e Publicações (GTP)**.

O objetivo é garantir que todos os desenvolvedores utilizem um ambiente padronizado, reduzindo problemas de configuração, incompatibilidades entre versões e diferenças de comportamento entre máquinas.

Ao final deste capítulo, o desenvolvedor deverá ser capaz de executar o projeto localmente, realizar alterações, executar testes e contribuir com segurança para a evolução do sistema.

2.2 Requisitos de Hardware

Embora o projeto possa ser executado em equipamentos mais modestos, recomenda-se a seguinte configuração mínima para uma experiência adequada.

Recurso	Recomendado
Processador	4 núcleos (Intel Core i5/Ryzen 5 ou superior)
Memória RAM	16 GB (mínimo de 8 GB)

Recurso	Recomendado
Armazenamento	SSD com pelo menos 30 GB livres
Resolução de tela	1920 × 1080 ou superior
Conexão com a Internet Necessária para dependências e integrações	
Para execução simultânea do frontend, backend, banco de dados, Docker e ferramentas de monitoramento, recomenda-se 16 GB ou mais de memória RAM.	

2.3 Sistemas Operacionais Suportados

O GTP é multiplataforma e poderá ser desenvolvido nos seguintes sistemas:

- Windows 11 (preferencialmente);
- Windows 10;
- Ubuntu LTS;
- Debian;
- Fedora;
- macOS (versões suportadas pela Apple).

As instruções deste manual utilizam como referência principal o Windows 11, mantendo compatibilidade com os demais sistemas operacionais.

2.4 Ferramentas Obrigatórias

Antes de iniciar o desenvolvimento, as seguintes ferramentas deverão estar instaladas.

Ferramenta	Versão Recomendada
Java	21 LTS
Maven	3.9 ou superior
Node.js	22 LTS ou superior
npm	Compatível com a versão do Node.js
Git	Versão estável mais recente

Ferramenta	Versão Recomendada
Docker Desktop	Última versão estável
Docker Compose	Integrado ao Docker Desktop
PostgreSQL	17
Visual Studio Code	Última versão estável

Sempre que possível, deverão ser utilizadas versões LTS (Long Term Support).

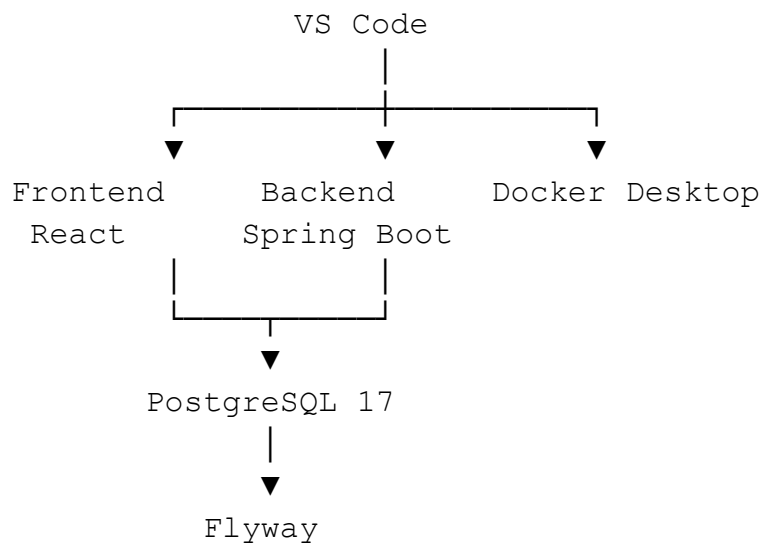
2.5 Ferramentas Opcionais

Embora não sejam obrigatórias, as ferramentas abaixo facilitam o desenvolvimento.

- Postman ou Insomnia (testes da API);
- DBeaver (administração do PostgreSQL);
- pgAdmin (administração do banco);
- GitHub Desktop (opcional para operações Git);
- Docker Extension para VS Code;
- Spring Boot Extension Pack;
- ESLint;
- Prettier.

2.6 Estrutura do Ambiente

A arquitetura local de desenvolvimento será semelhante ao ambiente de produção.



Essa padronização reduz diferenças entre os ambientes e facilita a identificação de problemas.

2.7 Clonagem do Repositório

O código-fonte deverá ser obtido a partir do repositório oficial do projeto.

Estrutura esperada:

```
gtp/
├── backend/
├── frontend/
├── docker/
├── docs/
├── scripts/
├── .github/
├── docker-compose.yml
├── README.md
└── LICENSE
```

Após a clonagem, recomenda-se verificar se todas as dependências foram obtidas corretamente.

2.8 Configuração do Backend

Antes da execução do backend deverão ser realizadas as seguintes etapas:

- instalar o Java 21;
- instalar o Maven;

- configurar a variável `JAVA_HOME`;
- verificar o funcionamento do Maven;
- configurar o arquivo `application-dev.yml`;
- definir as variáveis de ambiente necessárias.

O backend utilizará o perfil **dev** durante o desenvolvimento.

2.9 Configuração do Frontend

O frontend deverá ser preparado com os seguintes passos:

- instalar o Node.js;
- instalar as dependências do projeto;
- configurar o arquivo `.env.development`;
- definir a URL da API;
- verificar a execução do Vite.

Durante o desenvolvimento será utilizado o servidor local do Vite com recarregamento automático (*Hot Module Replacement*).

2.10 Configuração do Banco de Dados

O PostgreSQL poderá ser executado localmente ou por meio do Docker.

Configurações iniciais recomendadas:

Configuração Valor

Banco	<code>gtp_dev</code>
Porta	<code>5432</code>
Codificação	<code>UTF-8</code>
Time Zone	<code>UTC</code>
Usuário	Configurado por variável de ambiente
Senha	Configurada por variável de ambiente

As migrações serão aplicadas automaticamente pelo Flyway durante a inicialização da aplicação.

2.11 Configuração do Docker

O Docker Desktop deverá estar instalado e em execução antes da inicialização da infraestrutura.

Os principais serviços utilizados são:

- frontend;
- backend;
- PostgreSQL;
- Flyway;
- Prometheus;
- Grafana.

A orquestração local será realizada por meio do Docker Compose.

2.12 Configuração do Visual Studio Code

O VS Code será a IDE oficial recomendada para o projeto.

Extensões recomendadas:

Extensão	Finalidade
Extension Pack for Java	Desenvolvimento Spring Boot
Spring Boot Extension Pack	Ferramentas para Spring
Docker	Gerenciamento de contêineres
ESLint	Análise de JavaScript e TypeScript
Prettier	Formatação automática
GitLens	Histórico e colaboração Git
EditorConfig	Padronização de arquivos

Extensão	Finalidade
REST Client (opcional)	Testes de APIs diretamente no VS Code

Essas extensões aumentam a produtividade e mantêm o código consistente.

2.13 Variáveis de Ambiente

As configurações sensíveis deverão ser fornecidas por variáveis de ambiente.

Exemplos:

Backend:

```
DB_HOST
DB_PORT
DB_NAME
DB_USER
DB_PASSWORD
JWT_SECRET
MAIL_USERNAME
MAIL_PASSWORD
```

Frontend:

```
VITE_API_URL
VITE_APP_NAME
VITE_ENVIRONMENT
```

Nenhuma credencial deverá ser armazenada diretamente no código-fonte ou versionada no Git.

2.14 Verificação do Ambiente

Após a configuração, o desenvolvedor deverá confirmar:

- Java instalado corretamente;
- Maven funcionando;
- Node.js instalado;

- Docker em execução;
- PostgreSQL disponível;
- Flyway executando as migrações;
- frontend iniciando sem erros;
- backend iniciando sem erros;
- comunicação entre frontend e backend funcionando.

Essa validação reduz problemas nas etapas seguintes de desenvolvimento.

2.15 Solução de Problemas Comuns

Alguns problemas recorrentes podem ocorrer durante a configuração inicial.

Problema	Possível Solução
Porta 5432 ocupada	Alterar a porta ou encerrar o serviço conflitante.
Porta 8080 ocupada	Configurar uma nova porta para o backend.
Porta 5173 ocupada	Alterar a porta do Vite.
Java não encontrado	Verificar a variável JAVA_HOME.
Maven não reconhecido	Confirmar a configuração do PATH.
Docker não inicia	Verificar requisitos de virtualização do sistema operacional.
Falha nas migrações	Revisar os scripts do Flyway e a configuração do banco.

Sempre que possível, consultar os logs da aplicação para identificar a causa do problema.

2.16 Boas Práticas

Durante a preparação do ambiente recomenda-se:

- utilizar versões LTS das ferramentas;
- manter todas as dependências atualizadas;

- utilizar Docker para reduzir diferenças entre ambientes;
- evitar alterações manuais em arquivos compartilhados;
- documentar configurações específicas da equipe;
- revisar periodicamente as variáveis de ambiente.

2.17 Antipadrões

Devem ser evitados:

- utilizar versões diferentes das recomendadas sem validação;
- armazenar senhas no código-fonte;
- modificar configurações de produção durante o desenvolvimento;
- compartilhar arquivos .env contendo credenciais;
- executar a aplicação sem aplicar as migrações do banco;
- ignorar mensagens de erro na inicialização.

Esses antipadrões dificultam a colaboração e aumentam o risco de inconsistências.

2.18 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a configuração técnica do backend.
Documento 07 – Arquitetura Frontend	Especifica a estrutura e configuração do frontend.
Documento 09 – Docker	Detalha a utilização de Docker e Docker Compose.
Documento 10 – API REST	Define os serviços consumidos pelo frontend.
Documento 12 – Banco de Dados PostgreSQL	Especifica a configuração e administração do banco.

Documento	Relação
Documento 14 – Deploy e Infraestrutura	Define os ambientes e a infraestrutura utilizada durante o desenvolvimento.

2.19 Conclusão do Capítulo

A preparação adequada do ambiente de desenvolvimento é essencial para garantir produtividade e consistência durante a implementação do GTP. A padronização das ferramentas, versões e configurações reduz problemas de compatibilidade, facilita o trabalho em equipe e aproxima o ambiente local das condições encontradas em homologação e produção.

Com o ambiente configurado, o desenvolvedor estará apto a iniciar o desenvolvimento de novas funcionalidades, executar testes e contribuir para a evolução do projeto de forma segura e organizada.

3 – Estrutura do Projeto

3.1 Objetivo

Este capítulo apresenta a organização estrutural do código-fonte do **Gestor de Territórios e Publicações (GTP)**, descrevendo a divisão entre frontend, backend, infraestrutura e documentação.

A definição de uma estrutura padronizada facilita a navegação no projeto, melhora a manutenção do código, reduz o acoplamento entre componentes e permite que novos desenvolvedores compreendam rapidamente a organização da aplicação.

3.2 Princípios de Organização

A estrutura do projeto foi definida com base nos seguintes princípios:

- separação de responsabilidades (*Separation of Concerns*);
- modularização por domínio de negócio;
- baixo acoplamento;
- alta coesão;
- reutilização de componentes;

- facilidade de testes;
- escalabilidade;
- padronização.

Cada módulo deve possuir responsabilidades claramente definidas e depender apenas das camadas necessárias.

3.3 Estrutura Geral do Repositório

O repositório principal do GTP será organizado da seguinte forma:

```
gtp/
|
├── backend/
├── frontend/
├── docker/
├── database/
├── docs/
├── scripts/
├── .github/
|
├── docker-compose.yml
├── README.md
├── LICENSE
└── .gitignore
```

Descrição dos diretórios

Diretório Finalidade

backend Código da API REST em Spring Boot

frontend Interface web desenvolvida em React

docker Dockerfiles e arquivos auxiliares

database Scripts SQL, migrações e dados iniciais

docs Documentação técnica do projeto

scripts Scripts de automação

.github Workflows do GitHub Actions

3.4 Estrutura do Backend

O backend seguirá uma organização baseada em módulos e camadas.

```
backend/
├── src/
│   ├── main/
│   ├── java/
│   │   └── br/com/gtp/
│   ├── resources/
│   └── test/
├── pom.xml
└── Dockerfile
```

Dentro do pacote principal:

```
br/com/gtp/
config/
security/
common/
exception/
modules/
    usuarios/
    publicadores/
    territorios/
    publicacoes/
    designacoes/
    reunioes/
    notificacoes/
    relatorios/
```

Essa organização permite que cada domínio evolua de forma independente.

3.5 Estrutura Interna de um Módulo Backend

Cada módulo seguirá um padrão único.

Exemplo:

```
usuarios/  
controller/  
service/  
repository/  
entity/  
dto/  
mapper/  
validator/  
specification/  
exception/
```

Responsabilidade de cada camada

Camada	Responsabilidade
Controller	Endpoints REST
Service	Regras de negócio
Repository	Acesso aos dados
Entity	Entidades JPA
DTO	Objetos de transferência
Mapper	Conversão Entity ↔ DTO
Validator	Validações específicas
Specification	Consultas dinâmicas
Exception	Exceções do módulo

3.6 Estrutura do Frontend

O frontend será organizado por funcionalidades (*Feature-Based Architecture*), promovendo maior isolamento entre módulos.

```
frontend/  
  src/  
    assets/  
    components/  
    contexts/  
    hooks/  
    layouts/  
    pages/  
    routes/  
    services/  
    styles/  
    utils/  
    features/  
  App.jsx  
  main.jsx
```

3.7 Estrutura das Features

Cada funcionalidade possuirá seus próprios componentes, páginas, serviços e hooks.

Exemplo:

```
features/  
  usuarios/  
    components/  
    pages/  
    hooks/  
    services/  
    styles/  
    utils/
```

Esse modelo reduz dependências entre funcionalidades e facilita a manutenção.

3.8 Componentes Compartilhados

Os componentes reutilizáveis permanecerão na pasta components.

Exemplos:

```
components/  
  Button/  
  Card/  
  Modal/  
  DataTable/  
  Loading/  
  Pagination/  
  Sidebar/  
  Header/  
  Footer/  
  Form/  
  Input/  
  Select/  
  Dialog/  
  Toast/
```

Esses componentes poderão ser utilizados por qualquer módulo do sistema.

3.9 Organização dos Serviços

Toda comunicação com a API REST será centralizada na camada de serviços.

Exemplo:

```
services/  
  api.js  
  authService.js  
  usuarioService.js
```

```
territorioService.js  
publicadorService.js
```

Nenhuma chamada HTTP deverá ser realizada diretamente dentro dos componentes React.

3.10 Organização dos Estilos

A padronização visual será mantida por uma estrutura organizada de estilos.

```
styles/  
variables.css  
reset.css  
global.css  
typography.css  
animations.css  
themes/  
components/
```

Os estilos específicos das funcionalidades permanecerão dentro de cada *feature*.

3.11 Organização da Documentação

Toda a documentação do projeto permanecerá versionada.

```
docs/  
01-visao-geral/  
02-regras-negocio/  
03-casos-de-uso/  
...  
15-manual-desenvolvedor/
```

A documentação deverá evoluir em conjunto com o código-fonte.

3.12 Organização da Infraestrutura

Os arquivos relacionados à infraestrutura ficarão separados da aplicação.

```
docker/  
backend/  
frontend/  
nginx/  
prometheus/  
grafana/
```

Banco:

```
database/  
migration/  
seed/  
scripts/
```

Essa separação facilita a manutenção e a automação dos ambientes.

3.13 Convenções de Nomenclatura

Serão adotadas convenções padronizadas.

Classes Java

```
UsuarioService  
UsuarioController  
UsuarioRepository
```

Componentes React

```
UsuarioForm.jsx  
UsuarioCard.jsx  
DashboardCard.jsx
```

Hooks

```
useAuth  
usePagination
```

```
useUsuarios
```

Serviços

```
usuarioService.js
```

```
authService.js
```

Arquivos CSS

```
usuario.module.css
```

```
dashboard.css
```

A consistência na nomenclatura facilita a compreensão e reduz ambiguidades.

3.14 Organização dos Testes

Os testes acompanharão a estrutura do código.

Backend:

```
test/
```

```
unit/
```

```
integration/
```

```
repository/
```

```
service/
```

Frontend:

```
tests/
```

```
components/
```

```
pages/
```

```
hooks/
```

```
e2e/
```

Essa organização facilita a localização e manutenção dos testes automatizados.

3.15 Dependências Entre Camadas

A comunicação entre camadas deverá seguir a seguinte direção:

```
Controller
↓
Service
↓
Repository
↓
Banco de Dados
```

No frontend:

```
Page
↓
Component
↓
Hook
↓
Service
↓
API REST
```

Dependências inversas deverão ser evitadas.

3.16 Boas Práticas

Durante a organização do projeto recomenda-se:

- manter módulos independentes;
- evitar classes excessivamente grandes;
- separar responsabilidades;
- reutilizar componentes compartilhados;
- manter nomes consistentes;
- documentar módulos complexos;
- remover código obsoleto.

3.17 Antipadrões

Devem ser evitados:

- componentes gigantes;

- controllers contendo regras de negócio;
- acesso direto ao banco fora dos repositórios;
- chamadas HTTP diretamente em componentes React;
- mistura de estilos globais e locais sem critério;
- dependências circulares entre módulos;
- duplicação de código.

Esses antipadrões comprometem a manutenção e dificultam a evolução do sistema.

3.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura Geral	Define a visão arquitetural da solução.
Documento 06 – Arquitetura Backend	Especifica a organização do backend.
Documento 07 – Arquitetura Frontend	Define a arquitetura e os padrões do frontend.
Documento 09 – Docker	Organiza os artefatos de infraestrutura.
Documento 10 – API REST	Define a integração entre frontend e backend.
Documento 13 – Estratégia de Testes	Especifica a organização dos testes automatizados.

3.19 Conclusão do Capítulo

A estrutura do projeto foi concebida para proporcionar clareza, organização e facilidade de manutenção. A separação por camadas no backend e por funcionalidades no frontend permite que o sistema evolua de forma modular, reduzindo acoplamentos e facilitando a colaboração entre desenvolvedores.

A padronização da organização do código, da nomenclatura e da distribuição dos arquivos estabelece uma base sólida para o crescimento sustentável do GTP e simplifica o processo de desenvolvimento, testes e implantação.

4 – Padrões de Desenvolvimento

4.1 Objetivo

Este capítulo estabelece os padrões de desenvolvimento adotados no **Gestor de Territórios e Publicações (GTP)**, definindo convenções de codificação, organização do código, princípios arquiteturais e boas práticas que deverão ser seguidas por todos os colaboradores do projeto.

O objetivo é garantir consistência, legibilidade, manutenibilidade e qualidade do código ao longo de toda a evolução do sistema.

4.2 Princípios Gerais

Todo o desenvolvimento do GTP deverá seguir os seguintes princípios:

- simplicidade antes da complexidade;
- código legível e autoexplicativo;
- reutilização consciente;
- baixo acoplamento;
- alta coesão;
- responsabilidade única;
- modularização;
- desenvolvimento orientado a testes sempre que possível;
- documentação sincronizada com a implementação.

Esses princípios deverão orientar todas as decisões de projeto.

4.3 Clean Code

O projeto adotará as recomendações do **Clean Code**, buscando produzir código fácil de compreender e manter.

Diretrizes:

- utilizar nomes claros e descritivos;

- manter funções pequenas e objetivas;
- evitar duplicação de código;
- remover código morto;
- evitar comentários que expliquem código mal escrito;
- preferir métodos autoexplicativos;
- limitar responsabilidades por classe.

Sempre que um trecho exigir muitos comentários para ser entendido, deve-se avaliar sua refatoração.

4.4 Aplicação dos Princípios SOLID

A arquitetura do GTP será orientada pelos princípios SOLID.

Princípio	Aplicação no Projeto
S – Single Responsibility	Cada classe terá uma única responsabilidade.
O – Open/Closed	Componentes serão extensíveis sem necessidade de alteração frequente.
L – Liskov Substitution	Implementações poderão ser substituídas por abstrações compatíveis.
I – Interface Segregation	Interfaces serão específicas e coesas.
D – Dependency Inversion	Dependências serão estabelecidas por abstrações sempre que possível.

A aplicação consistente desses princípios reduz o acoplamento e facilita a evolução do sistema.

4.5 Convenções para Java

As seguintes convenções deverão ser utilizadas no backend.

Classes

Utilizar **PascalCase**.

Exemplos:

```
UsuarioController  
PublicadorService  
TerritorioRepository  
RelatorioMapper
```

Métodos

Utilizar **camelCase**.

```
buscarPorId()  
salvar()  
atualizarPublicador()  
listarTerritoriosDisponiveis()
```

Variáveis

```
usuarioLogado  
quantidadeTerritorios  
nomePublicador
```

Constantes

```
MAX_TENTATIVAS  
JWT_EXPIRATION  
DEFAULT_PAGE_SIZE
```

4.6 Convenções para React

Os componentes React seguirão as seguintes regras.

Componentes

Sempre em **PascalCase**.

```
DashboardCard.jsx  
TerritorioForm.jsx  
PublicadorList.jsx
```

Hooks

Sempre iniciados por use.

```
useAuth()  
usePagination()  
useTerritorios()
```

Contexts

```
AuthContext  
ThemeContext  
NotificationContext
```

Arquivos de serviço

```
usuarioService.js  
territorioService.js  
api.js
```

4.7 Convenções para Banco de Dados

A modelagem seguirá padrões consistentes.

Tabelas

```
usuarios  
publicadores  
territorios  
publicacoes
```

Colunas

```
id  
nome  
email  
data_criacao  
ultima_atualizacao
```

Chaves estrangeiras

```
usuario_id  
territorio_id  
publicador_id
```

Nomes deverão ser claros e evitar abreviações desnecessárias.

4.8 Organização dos Métodos

Os métodos deverão ser curtos e possuir responsabilidades bem definidas.

Recomendações:

- uma única responsabilidade por método;
- preferencialmente até 30 linhas (quando razoável);
- evitar múltiplos níveis de aninhamento;
- extrair métodos auxiliares quando necessário;
- utilizar retornos antecipados (*guard clauses*) para reduzir complexidade.

4.9 Tratamento de Exceções

O tratamento de erros deverá ser centralizado.

Diretrizes:

- utilizar exceções específicas;
- evitar capturas genéricas (Exception);
- registrar erros relevantes em logs;
- fornecer mensagens claras para a API;
- não ocultar exceções.

O backend utilizará um mecanismo global de tratamento de exceções por meio de `@RestControllerAdvice`.

4.10 Validação de Dados

As validações deverão ocorrer em múltiplas camadas.

Camada	Responsabilidade
Frontend	Validação inicial da entrada do usuário.
Backend	Validação das regras de negócio e integridade dos dados.
Banco de Dados	Restrições estruturais e integridade referencial.

Essa abordagem reduz inconsistências e melhora a experiência do usuário.

4.11 Comentários e Documentação

Comentários deverão ser utilizados apenas quando agregarem valor.

Recomendações:

- explicar decisões arquiteturais;
- documentar algoritmos complexos;
- evitar comentários redundantes;
- manter comentários atualizados.

Sempre que possível, o próprio código deverá ser suficientemente claro para dispensar explicações adicionais.

4.12 Organização dos Imports

Os imports deverão permanecer organizados.

No Java:

- bibliotecas padrão;
- bibliotecas de terceiros;
- classes do projeto.

No React:

- React;
- bibliotecas externas;

- componentes compartilhados;
- componentes locais;
- estilos.

Essa organização melhora a legibilidade e reduz conflitos.

4.13 Controle de Dependências

A inclusão de novas dependências deverá observar os seguintes critérios:

- necessidade comprovada;
- compatibilidade com o projeto;
- manutenção ativa da biblioteca;
- licença compatível;
- baixo impacto no tamanho da aplicação;
- ausência de funcionalidades já disponíveis em outras dependências.

Toda nova dependência deverá ser justificada e documentada.

4.14 Refatoração

A refatoração deverá ser um processo contínuo.

Situações que indicam necessidade de refatoração:

- duplicação de código;
- métodos excessivamente longos;
- classes com muitas responsabilidades;
- complexidade elevada;
- dificuldade de testes;
- baixa legibilidade.

Toda refatoração deverá preservar o comportamento funcional da aplicação e ser acompanhada por testes.

4.15 Boas Práticas

Durante o desenvolvimento deverão ser observadas as seguintes recomendações:

- escrever código simples;
- utilizar nomes significativos;
- reutilizar componentes e serviços;
- remover código não utilizado;
- manter funções pequenas;
- revisar o código antes do commit;
- manter cobertura adequada de testes;
- atualizar a documentação quando necessário.

4.16 Antipadrões

Devem ser evitados:

- métodos excessivamente longos;
- classes com múltiplas responsabilidades;
- duplicação de código (*code duplication*);
- números mágicos (*magic numbers*);
- comentários desatualizados;
- dependências circulares;
- uso excessivo de variáveis globais;
- tratamento genérico de exceções;
- lógica de negócio em componentes React ou controllers.

Esses antipadrões dificultam a manutenção, aumentam a complexidade e reduzem a qualidade do sistema.

4.17 Checklist de Desenvolvimento

Antes de concluir uma funcionalidade, o desenvolvedor deverá verificar:

- código segue os padrões definidos;
- nomenclatura consistente;
- ausência de código duplicado;
- tratamento adequado de erros;
- validações implementadas;
- testes atualizados;
- documentação revisada;
- ausência de *warnings* relevantes;
- revisão pessoal realizada antes do commit.

Esse checklist reduz retrabalho e aumenta a qualidade das entregas.

4.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura Geral	Define os princípios arquiteturais aplicados ao código.
Documento 06 – Arquitetura Backend	Especifica os padrões do backend.
Documento 07 – Arquitetura Frontend	Define os padrões do frontend.
Documento 10 – API REST	Estabelece convenções para implementação da API.
Documento 13 – Estratégia de Testes	Complementa as diretrizes de qualidade e validação.
Documento 15 – Manual do Desenvolvedor	Consolida as práticas de desenvolvimento utilizadas no projeto.

4.19 Conclusão do Capítulo

A adoção de padrões de desenvolvimento consistentes é essencial para garantir a qualidade e a evolução sustentável do GTP. A aplicação de princípios como Clean Code, SOLID, modularização e padronização de nomenclaturas facilita a colaboração entre desenvolvedores, reduz a ocorrência de defeitos e torna o sistema mais simples de compreender e manter.

Essas diretrizes deverão ser seguidas em todas as contribuições para assegurar que o código permaneça coeso, testável e alinhado à arquitetura definida para o projeto.

5 – Fluxo de Trabalho (Git e GitHub)

5.1 Objetivo

Este capítulo define o fluxo oficial de desenvolvimento colaborativo do **Gestor de Territórios e Publicações (GTP)** utilizando **Git** e **GitHub**.

O objetivo é estabelecer um processo padronizado para controle de versão, desenvolvimento de funcionalidades, correção de defeitos, revisão de código e integração contínua, garantindo rastreabilidade, qualidade e estabilidade da aplicação.

5.2 Princípios do Fluxo de Trabalho

O fluxo de trabalho do projeto baseia-se nos seguintes princípios:

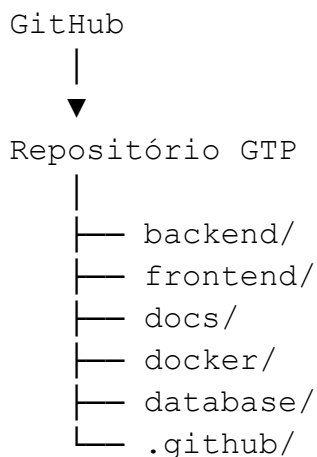
- desenvolvimento incremental;
- commits pequenos e frequentes;
- integração contínua;
- revisão de código;
- rastreabilidade das alterações;
- automação de testes;
- facilidade de rollback;
- estabilidade da branch principal.

Nenhuma alteração deverá ser realizada diretamente na branch principal sem passar pelo fluxo definido.

5.3 Repositório Oficial

Todo o código-fonte do GTP será mantido em um repositório Git hospedado no GitHub.

Estrutura simplificada:



O repositório concentra código, documentação, workflows, scripts e histórico completo do projeto.

5.4 Estratégia de Branches

O projeto adotará um modelo inspirado no **GitHub Flow**, mantendo uma estrutura simples e adequada ao desenvolvimento contínuo.

Branches principais

Branch	Finalidade
--------	------------

<code>main</code>	Código estável e pronto para produção.
-------------------	--

<code>develop</code>	Integração contínua das funcionalidades em desenvolvimento.
----------------------	---

Branches temporárias

Prefixo	Utilização
---------	------------

<code>feature/</code>	Desenvolvimento de novas funcionalidades.
-----------------------	---

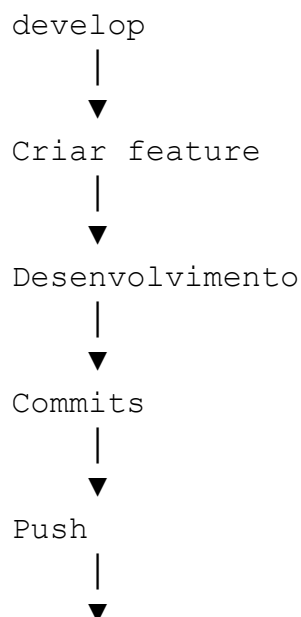
Prefixo	Utilização
bugfix/	Correção de defeitos identificados durante o desenvolvimento.
hotfix/	Correções urgentes aplicadas à produção.
release/	Preparação de uma nova versão para implantação.
docs/	Atualizações da documentação.
refactor/	Refatorações sem alteração de comportamento funcional.
test/	Implementação ou ajustes de testes automatizados.

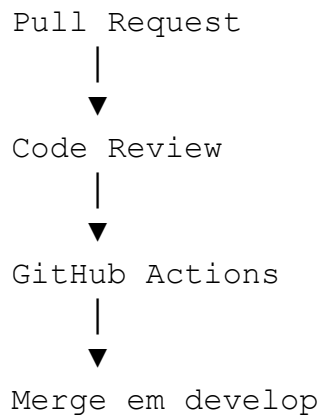
Exemplos:

```
feature/cadastro-publicadores
feature/dashboard-indicadores
bugfix/login-jwt
hotfix/corrigir-token-expirado
docs/manual-desenvolvedor
refactor/service-publicador
test/api-publicadores
```

5.5 Fluxo de Desenvolvimento

O desenvolvimento de uma funcionalidade seguirá as etapas abaixo.





Após a aprovação e validação dos testes, a funcionalidade poderá ser integrada à branch develop.

5.6 Convenção de Commits

As mensagens de commit deverão seguir um padrão inspirado no **Conventional Commits**.

Prefixo	Finalidade
---------	------------

feat:	Nova funcionalidade
-------	---------------------

fix:	Correção de defeito
------	---------------------

docs:	Alteração na documentação
-------	---------------------------

style:	Ajustes de formatação
--------	-----------------------

refactor:	Refatoração
-----------	-------------

test:	Inclusão ou alteração de testes
-------	---------------------------------

chore:	Tarefas de manutenção
--------	-----------------------

build:	Alterações de build ou dependências
--------	-------------------------------------

ci:	Ajustes de integração contínua
-----	--------------------------------

Exemplos:

```
feat: implementar cadastro de territórios
```

```
fix: corrigir validação do login
```

```
docs: atualizar documento da API REST
```

```
refactor: simplificar serviço de publicadores  
test: adicionar testes da API de usuários  
ci: atualizar workflow de deploy
```

As mensagens deverão ser claras, objetivas e descrever a principal alteração realizada.

5.7 Pull Requests

Toda alteração deverá ser submetida por meio de um **Pull Request (PR)**.

O PR deverá conter:

- título descritivo;
- resumo das alterações;
- justificativa da implementação;
- impacto esperado;
- evidências de testes realizados;
- referência à issue correspondente, quando aplicável.

Alterações significativas deverão incluir capturas de tela, diagramas ou exemplos de uso quando pertinentes.

5.8 Code Review

Todo Pull Request deverá passar por revisão antes da integração.

Durante a revisão serão observados aspectos como:

- aderência aos padrões do projeto;
- legibilidade do código;
- arquitetura;
- tratamento de erros;
- segurança;
- cobertura de testes;
- desempenho;

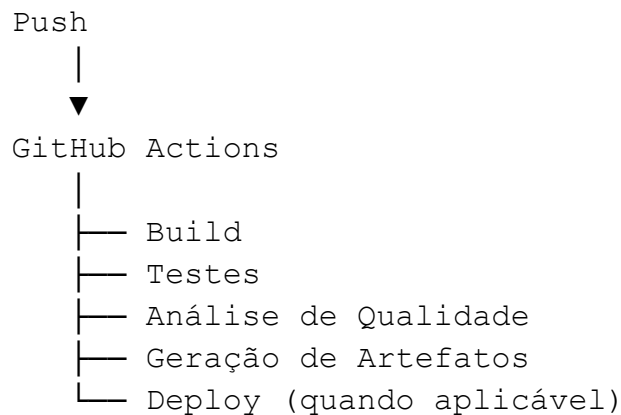
- documentação.

Sempre que possível, o feedback deverá ser construtivo e fundamentado.

5.9 GitHub Actions

O projeto utilizará **GitHub Actions** para automatizar tarefas do processo de desenvolvimento.

Fluxo simplificado:



A automação reduz erros manuais e garante maior confiabilidade nas integrações.

5.10 Integração Contínua (CI)

A pipeline de Integração Contínua deverá executar, no mínimo:

- compilação do backend;
- instalação das dependências do frontend;
- execução dos testes automatizados;
- análise estática de código;
- validação das migrações do banco;
- verificação da documentação.

A integração somente deverá ocorrer quando todas as etapas forem concluídas com sucesso.

5.11 Versionamento

O GTP utilizará **Versionamento Semântico (Semantic Versioning)**.

Formato:

MAJOR.MINOR.PATCH

Exemplos:

1.0.0

1.1.0

1.2.5

2.0.0

Regras:

- **MAJOR:** mudanças incompatíveis;
- **MINOR:** novas funcionalidades compatíveis;
- **PATCH:** correções sem alteração de compatibilidade.

5.12 Releases

Cada versão oficial deverá possuir uma *Release* no GitHub contendo:

- número da versão;
- data da publicação;
- funcionalidades adicionadas;
- correções realizadas;
- melhorias implementadas;
- problemas conhecidos (quando houver).

Essa prática facilita a rastreabilidade das evoluções do projeto.

5.13 Gestão de Issues

As funcionalidades, melhorias e defeitos deverão ser registrados como **Issues**.

Cada issue deverá conter:

- título claro;
- descrição detalhada;

- prioridade;
- responsável;
- critérios de aceite;
- etiquetas (*labels*);
- marco (*milestone*), quando aplicável.

As issues serão a principal referência para o planejamento do desenvolvimento.

5.14 Boas Práticas

Durante o trabalho com Git e GitHub recomenda-se:

- realizar commits pequenos e frequentes;
- sincronizar a branch regularmente com develop;
- escrever mensagens de commit descritivas;
- revisar o código antes do push;
- manter Pull Requests objetivos;
- resolver conflitos localmente antes da revisão;
- excluir branches temporárias após o merge.

5.15 Antipadrões

Devem ser evitados:

- commits muito grandes;
- commits contendo múltiplas funcionalidades distintas;
- envio de código sem testes básicos;
- merge direto na branch main;
- mensagens de commit genéricas como "ajustes" ou "correções";
- Pull Requests excessivamente extensos;
- branches desatualizadas por longos períodos.

Esses antipadrões dificultam a revisão, aumentam riscos e comprometem a rastreabilidade.

5.16 Checklist Antes do Merge

Antes de aprovar um Pull Request, deverá ser verificado:

- código compilando corretamente;
- testes executados com sucesso;
- padrões de desenvolvimento respeitados;
- documentação atualizada;
- ausência de conflitos;
- revisão concluída;
- aprovação obtida;
- pipeline CI executada com sucesso.

Esse checklist contribui para manter a estabilidade da branch principal.

5.17 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a organização do código versionado.
Documento 07 – Arquitetura Frontend	Complementa os padrões de organização das funcionalidades.
Documento 09 – Docker	Especifica os arquivos de infraestrutura mantidos no repositório.
Documento 13 – Estratégia de Testes	Define os testes executados durante a integração contínua.
Documento 14 – Deploy e Infraestrutura	Especifica as pipelines de CI/CD e os processos de implantação.
Documento 15 – Manual do Desenvolvedor	Consolida o fluxo de colaboração adotado no projeto.

5.18 Conclusão do Capítulo

O fluxo de trabalho definido para o GTP estabelece um processo organizado, seguro e rastreável para o desenvolvimento colaborativo. A utilização de Git, GitHub, Pull Requests, Code Review e GitHub Actions garante que todas as alterações sejam revisadas, testadas e integradas de forma controlada.

A adoção de convenções de branches, mensagens de commit e versionamento semântico contribui para a manutenção da qualidade do código, facilita a colaboração entre os membros da equipe e prepara o projeto para um processo contínuo de evolução.

6 – Execução do Projeto

6.1 Objetivo

Este capítulo apresenta os procedimentos necessários para executar o **Gestor de Territórios e Publicações (GTP)** em ambiente local de desenvolvimento.

São descritos os processos de inicialização do backend, frontend, banco de dados, infraestrutura Docker e ferramentas auxiliares, permitindo que qualquer desenvolvedor configure e execute o projeto de forma padronizada.

6.2 Pré-requisitos

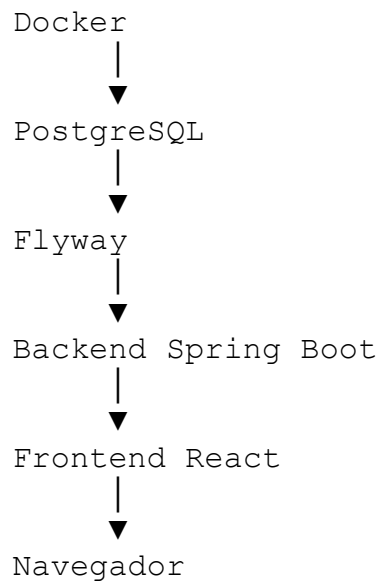
Antes da execução do projeto, deverão estar configurados:

- Java 21 LTS;
- Maven;
- Node.js LTS;
- npm;
- PostgreSQL 17 ou Docker Desktop;
- Git;
- Visual Studio Code;
- variáveis de ambiente;
- repositório clonado.

Além disso, recomenda-se validar a instalação utilizando os comandos apropriados para verificar as versões das ferramentas.

6.3 Estrutura de Inicialização

A sequência recomendada para inicialização dos serviços é:



Essa ordem garante que os serviços dependentes estejam disponíveis antes da inicialização da aplicação.

6.4 Execução do Banco de Dados

O PostgreSQL poderá ser iniciado de duas formas.

Opção 1 – Docker

A forma recomendada é utilizar o ambiente Docker Compose.

Serviços iniciados:

- PostgreSQL;
- Flyway;
- Prometheus;
- Grafana (quando necessário).

Opção 2 – Instalação Local

Caso o Docker não seja utilizado:

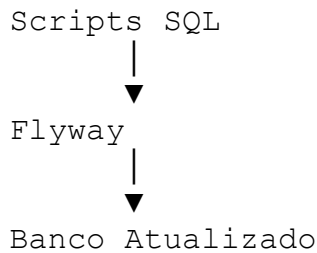
- iniciar o serviço PostgreSQL;
- verificar a existência do banco `gtp_dev`;

- validar usuário e senha configurados;
- confirmar disponibilidade da porta configurada.

6.5 Execução das Migrações Flyway

Durante a inicialização do backend, o Flyway executará automaticamente todas as migrações pendentes.

Fluxo:



Caso ocorra falha em alguma migração, a execução da aplicação será interrompida até que o problema seja corrigido.

6.6 Inicialização do Backend

O backend será iniciado utilizando o perfil de desenvolvimento.

Durante a inicialização, deverão ser observados os seguintes pontos:

- carregamento do Spring Boot;
- conexão com o PostgreSQL;
- execução das migrações;
- criação do contexto da aplicação;
- inicialização do servidor web.

Ao final do processo, a API REST deverá estar disponível para consumo pelo frontend.

6.7 Inicialização do Frontend

Após o backend estar disponível, o frontend poderá ser iniciado.

Durante a inicialização serão executadas as seguintes etapas:

- instalação das dependências (quando necessário);
- carregamento do Vite;
- compilação dos componentes;
- ativação do Hot Module Replacement (HMR);

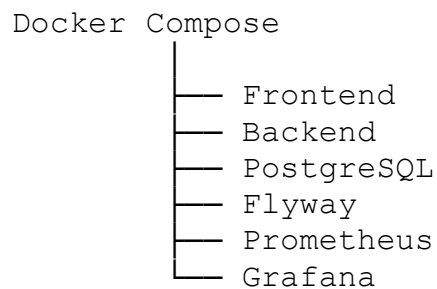
- abertura do servidor local.

O frontend deverá conseguir estabelecer comunicação com a API REST configurada nas variáveis de ambiente.

6.8 Execução Utilizando Docker Compose

A execução completa da aplicação poderá ocorrer por meio do Docker Compose.

Arquitetura simplificada:



Essa abordagem aproxima o ambiente de desenvolvimento da infraestrutura utilizada em produção.

6.9 Perfis de Execução

A aplicação utilizará perfis específicos para cada ambiente.

Perfil	Finalidade
dev	Desenvolvimento local
test	Execução de testes automatizados
staging	Homologação
prod	Produção

Cada perfil possuirá configurações próprias de banco de dados, logs, autenticação e integração.

6.10 Variáveis de Ambiente

Antes da execução deverão estar configuradas todas as variáveis necessárias.

Exemplos para o backend:

- `DB_HOST;`

- `DB_PORT;`
- `DB_NAME;`
- `DB_USER;`
- `DB_PASSWORD;`
- `JWT_SECRET;`
- `SPRING_PROFILES_ACTIVE.`

Exemplos para o frontend:

- `VITE_API_URL;`
- `VITE_APP_NAME;`
- `VITE_ENVIRONMENT.`

As credenciais nunca deverão ser armazenadas diretamente no código-fonte.

6.11 Validação da Inicialização

Após todos os serviços estarem em execução, deverão ser realizadas as seguintes verificações:

- backend iniciado sem erros;
- frontend carregado corretamente;
- banco conectado;
- migrações aplicadas;
- autenticação funcionando;
- comunicação entre frontend e backend;
- logs sem erros críticos.

Caso alguma etapa falhe, a investigação deverá iniciar pelos logs do componente correspondente.

6.12 Logs Durante a Execução

Durante o desenvolvimento, recomenda-se acompanhar continuamente os logs da aplicação.

Fontes principais:

Componente	Finalidade
Backend	Exceções, consultas, autenticação e regras de negócio
Frontend	Erros JavaScript e comunicação com a API
PostgreSQL	Consultas e conexões
Docker	Estado dos contêineres

Componente	Finalidade
Flyway	Migrações executadas

Os logs são a principal fonte de diagnóstico para problemas de execução.

6.13 Debug da Aplicação

O ambiente deverá permitir depuração tanto do frontend quanto do backend.

Backend:

- breakpoints;
- inspeção de variáveis;
- depuração de requisições REST;
- análise das exceções.

Frontend:

- DevTools do navegador;
- React Developer Tools;
- inspeção da árvore de componentes;
- análise do estado da aplicação.

A depuração deverá ser utilizada antes de aplicar correções em defeitos.

6.14 Execução dos Testes

Antes de concluir qualquer alteração, recomenda-se executar:

- testes unitários;
- testes de integração;
- testes da API;
- testes do frontend;
- verificações de lint;
- validações de formatação.

A execução frequente dos testes reduz a introdução de regressões.

6.15 Encerramento da Aplicação

Ao finalizar o desenvolvimento, recomenda-se:

1. encerrar o frontend;
2. interromper o backend;
3. finalizar os contêineres Docker, quando utilizados;
4. verificar se não permanecem processos em execução;
5. confirmar que conexões com o banco foram encerradas corretamente.

Esse procedimento evita consumo desnecessário de recursos do ambiente local.

6.16 Solução de Problemas Frequentes

Alguns problemas comuns durante a execução incluem:

Problema	Possível Causa
Backend não inicia	Erro de configuração, banco indisponível ou falha nas migrações
Frontend não conecta	URL da API incorreta ou backend indisponível
Erro de autenticação	Token inválido ou configuração de segurança incorreta
Falha no Flyway	Migração inconsistente ou banco incompatível
Docker não sobe	Contêiner em conflito ou portas ocupadas
Porta em uso	Outro serviço utilizando a mesma porta

A análise dos logs normalmente permite identificar rapidamente a origem da falha.

6.17 Boas Práticas

Durante a execução do projeto recomenda-se:

- iniciar os serviços na ordem definida;
- acompanhar os logs constantemente;
- manter as dependências atualizadas;
- utilizar perfis corretos para cada ambiente;
- validar alterações antes de realizar commits;
- executar os testes frequentemente;
- utilizar Docker para manter ambientes consistentes.

6.18 Antipadrões

Devem ser evitados:

- executar a aplicação sem aplicar as migrações;
- alterar configurações diretamente em produção;
- utilizar credenciais reais em ambiente de desenvolvimento;
- ignorar mensagens de erro durante a inicialização;
- manter processos antigos ocupando portas utilizadas pela aplicação;

- executar versões incompatíveis das ferramentas.

Esses antipadrões aumentam a ocorrência de falhas e dificultam a manutenção.

6.19 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a inicialização e configuração do backend.
Documento 07 – Arquitetura Frontend	Especifica a execução da aplicação React.
Documento 09 – Docker	Detalha a execução utilizando contêineres.
Documento 10 – API REST	Define os serviços disponibilizados pelo backend.
Documento 12 – Banco de Dados PostgreSQL	Especifica a configuração do banco e das migrações.
Documento 14 – Deploy e Infraestrutura	Define os ambientes e a infraestrutura utilizada durante a execução.

6.20 Conclusão do Capítulo

A execução padronizada do GTP garante que todos os desenvolvedores trabalhem em condições semelhantes, reduzindo inconsistências entre ambientes e facilitando a identificação de problemas. A utilização de Docker, Flyway, Spring Boot e Vite proporciona um fluxo de inicialização previsível e alinhado às práticas modernas de desenvolvimento.

A validação do ambiente, o acompanhamento dos logs e a execução frequente dos testes contribuem para aumentar a qualidade do software e reduzir o tempo gasto na resolução de falhas.

7 – Debug, Logs e Solução de Problemas

7.1 Objetivo

Este capítulo apresenta as práticas, ferramentas e procedimentos para depuração (*debug*), análise de logs e resolução de problemas no **Gestor de Territórios e Publicações (GTP)**.

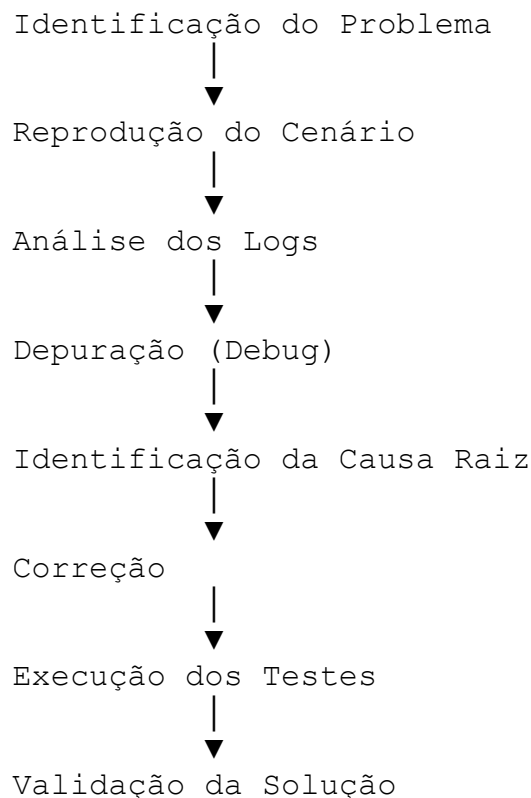
O objetivo é fornecer um guia para que os desenvolvedores consigam identificar rapidamente a origem de falhas, compreender o comportamento da aplicação e aplicar

correções de forma eficiente, reduzindo o tempo de diagnóstico e aumentando a confiabilidade do sistema.

7.2 Estratégia de Diagnóstico

A investigação de problemas deverá seguir um fluxo estruturado, evitando alterações sem análise prévia.

Fluxo recomendado:



Sempre que possível, a causa raiz deve ser identificada antes da implementação de qualquer correção.

7.3 Ferramentas de Debug

As seguintes ferramentas são recomendadas para o desenvolvimento do GTP.

Ferramenta	Finalidade
Visual Studio Code	Depuração do frontend e backend
Java Debugger	Depuração da aplicação Spring Boot
Chrome DevTools	Inspeção da interface e execução JavaScript

Ferramenta	Finalidade
React Developer Tools	Inspeção da árvore de componentes React
Postman / Insomnia	Testes e depuração da API REST
DBeaver / pgAdmin	Consulta e inspeção do banco PostgreSQL
Docker Desktop	Análise de contêineres e logs

Essas ferramentas permitem investigar problemas em diferentes camadas da aplicação.

7.4 Depuração do Backend

O backend poderá ser executado em modo de depuração para permitir:

- utilização de *breakpoints*;
- inspeção de variáveis;
- acompanhamento da pilha de chamadas (*call stack*);
- avaliação de expressões;
- monitoramento do fluxo de execução.

Recomenda-se iniciar a depuração a partir do ponto onde o problema é reproduzido, evitando sessões muito amplas.

7.5 Depuração do Frontend

No frontend, a depuração poderá ser realizada utilizando:

- DevTools do navegador;
- React Developer Tools;
- inspeção de componentes;
- análise do estado (*state*);
- monitoramento do Context API;
- inspeção de chamadas HTTP;
- análise do armazenamento local (*Local Storage* e *Session Storage*).

O uso de pontos de interrupção e do console do navegador auxilia na identificação de erros de lógica e integração.

7.6 Análise de Logs

Os logs são a principal fonte de informação para investigação de falhas.

Durante o desenvolvimento, devem ser monitorados:

Origem	Informações Disponíveis
Backend	Inicialização, autenticação, exceções, regras de negócio e acesso ao banco
Frontend	Erros JavaScript, chamadas HTTP e renderização
PostgreSQL	Conexões, consultas e erros de banco
Docker	Estado dos contêineres e serviços
Flyway	Execução das migrações
GitHub Actions	Build, testes e deploy

A análise conjunta dos logs permite identificar problemas de integração entre os componentes.

7.7 Níveis de Log

A aplicação utilizará níveis de log padronizados.

Nível	Utilização
TRACE	Diagnóstico extremamente detalhado
DEBUG	Informações para desenvolvimento
INFO	Eventos normais da aplicação
WARN	Situações inesperadas que não interrompem o funcionamento
ERROR	Falhas que impedem a execução de uma operação

Em ambiente de desenvolvimento recomenda-se utilizar os níveis `DEBUG` e `INFO`, enquanto em produção deve-se restringir os logs ao nível necessário para evitar excesso de informações.

7.8 Problemas Frequentes

Algumas falhas recorrentes durante o desenvolvimento incluem:

Problema	Possíveis Causas
Backend não inicia	Banco indisponível, erro de configuração ou falha nas migrações
Frontend não carrega	Dependências não instaladas ou erro de compilação
Erro de CORS	Configuração incorreta da política de segurança
Token JWT inválido	Expiração ou configuração incorreta da autenticação
Falha de conexão com o banco	Credenciais ou endereço incorretos

Problema	Possíveis Causas
Erro de migração Flyway	Script inconsistente ou banco fora da versão esperada
Porta ocupada	Outro processo utilizando a mesma porta

Esses problemas devem ser investigados inicialmente por meio dos logs correspondentes.

7.9 Tratamento de Exceções

O backend utilizará um mecanismo global de tratamento de exceções.

Diretrizes:

- capturar exceções específicas;
- registrar informações relevantes em log;
- retornar respostas padronizadas para a API;
- evitar exposição de detalhes internos ao usuário.

A padronização facilita o diagnóstico e melhora a experiência do consumidor da API.

7.10 Diagnóstico de Problemas de Banco de Dados

Ao investigar falhas relacionadas ao PostgreSQL, recomenda-se verificar:

- disponibilidade do serviço;
- conexão estabelecida;
- credenciais utilizadas;
- execução das migrações Flyway;
- integridade das tabelas;
- restrições e índices;
- desempenho das consultas.

Ferramentas como DBeaver ou pgAdmin auxiliam na inspeção do banco durante o desenvolvimento.

7.11 Diagnóstico de Problemas de API

Ao analisar falhas na API REST, verificar:

- código HTTP retornado;
- corpo da resposta;
- cabeçalhos enviados;

- autenticação;
- autorização;
- validação dos dados;
- logs do backend.

Ferramentas como Postman ou Insomnia facilitam a reprodução das requisições.

7.12 Diagnóstico de Problemas no Frontend

Problemas comuns incluem:

- falhas de renderização;
- estados inconsistentes;
- chamadas HTTP incorretas;
- erros de roteamento;
- problemas de autenticação;
- componentes reutilizados incorretamente.

A inspeção dos componentes React e do tráfego de rede permite localizar rapidamente a origem da maioria dessas falhas.

7.13 Estratégia de Correção

Após identificar a causa do problema, recomenda-se:

1. implementar a correção;
2. executar os testes relacionados;
3. validar o comportamento esperado;
4. revisar impactos em outros módulos;
5. registrar a alteração por meio de commit;
6. atualizar a documentação, quando necessário.

Correções emergenciais devem ser tratadas por meio do fluxo de *hotfix* definido no projeto.

7.14 Boas Práticas

Durante o processo de depuração recomenda-se:

- reproduzir o problema antes de alterar o código;
- utilizar *breakpoints* em vez de excesso de mensagens de console;
- analisar a causa raiz;
- manter os logs organizados;

- remover logs temporários antes do commit;
- registrar informações úteis para futuras investigações;
- validar a solução em diferentes cenários.

7.15 Antipadrões

Devem ser evitados:

- corrigir sintomas sem identificar a causa;
- utilizar `System.out.println()` ou `console.log()` de forma permanente;
- ignorar mensagens de erro;
- ocultar exceções;
- remover validações para "resolver" problemas;
- alterar múltiplos módulos sem necessidade;
- realizar correções sem testes.

Esses antipadrões dificultam a manutenção e aumentam o risco de regressões.

7.16 Checklist de Depuração

Antes de concluir uma investigação, verificar:

- problema reproduzido;
- causa raiz identificada;
- correção implementada;
- testes executados;
- logs revisados;
- ausência de novos erros;
- documentação atualizada, quando aplicável;
- commit preparado conforme o padrão do projeto.

7.17 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a estrutura interna da aplicação e o tratamento de exceções.
Documento 07 – Arquitetura Frontend	Complementa a depuração da interface e dos componentes React.
Documento 10 – API REST	Especifica os padrões de respostas e erros da API.
Documento 12 – Banco de Dados PostgreSQL	Define a estrutura e a administração do banco de dados.

Documento	Relação
Documento 13 – Estratégia de Testes	Estabelece os testes utilizados para validar correções.
Documento 14 – Deploy e Infraestrutura	Especifica monitoramento, observabilidade e gerenciamento de logs em produção.

7.18 Conclusão do Capítulo

A utilização de um processo estruturado de depuração e análise de logs permite identificar falhas de maneira mais rápida e precisa. O uso adequado das ferramentas de desenvolvimento, aliado a uma estratégia consistente de tratamento de exceções e validação por testes, reduz o tempo de resolução de problemas e aumenta a qualidade do software.

Ao seguir as diretrizes apresentadas neste capítulo, os desenvolvedores estarão preparados para investigar incidentes de forma sistemática, preservar a estabilidade da aplicação e contribuir para a evolução contínua do **Gestor de Territórios e Publicações (GTP)**.

8 – Testes e Qualidade

8.1 Objetivo

Este capítulo estabelece as diretrizes para a implementação, execução e manutenção dos testes automatizados e dos processos de garantia da qualidade no **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é assegurar que todas as funcionalidades sejam desenvolvidas com elevado nível de confiabilidade, reduzindo regressões e garantindo que o sistema permaneça estável durante toda a sua evolução.

8.2 Princípios da Qualidade

A qualidade do software será tratada como responsabilidade de toda a equipe de desenvolvimento.

Os principais princípios adotados são:

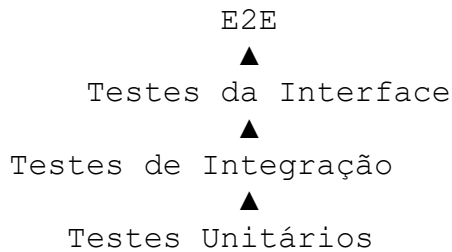
- prevenção de defeitos;
- automação de testes;
- integração contínua;

- revisão de código;
- rastreabilidade;
- melhoria contínua;
- documentação atualizada;
- validação antes da entrega.

A qualidade deverá ser considerada desde o início da implementação de cada funcionalidade.

8.3 Pirâmide de Testes

O GTP adotará a estratégia clássica da **Pirâmide de Testes**, priorizando testes rápidos e de fácil manutenção.



Distribuição recomendada:

Tipo	Percentual aproximado
Testes Unitários	70%
Testes de Integração	20%
Testes E2E	10%

Essa distribuição busca equilíbrio entre cobertura, velocidade de execução e custo de manutenção.

8.4 Testes Unitários

Os testes unitários verificam o comportamento isolado de classes, métodos e funções.

Backend

Serão testados:

- Services;
- Validators;

- Mappers;
- Utilitários;
- Regras de negócio.

Frontend

Serão testados:

- Hooks;
- Funções utilitárias;
- Componentes isolados;
- Contexts;
- Lógica de formulários.

Os testes unitários deverão ser independentes, rápidos e determinísticos.

8.5 Testes de Integração

Os testes de integração validam a comunicação entre diferentes componentes da aplicação.

Exemplos:

- Controller ↔ Service;
- Service ↔ Repository;
- Repository ↔ PostgreSQL;
- API ↔ Banco de Dados;
- Frontend ↔ Backend.

Esses testes garantem que os módulos funcionem corretamente em conjunto.

8.6 Testes da API REST

A API REST deverá possuir testes para validar:

- autenticação;
- autorização;
- validação de dados;
- códigos HTTP;
- estrutura das respostas;
- paginação;
- filtros;
- tratamento de erros.

Cada endpoint deverá possuir cenários positivos e negativos.

8.7 Testes do Frontend

A interface deverá ser validada por meio de testes automatizados que verifiquem:

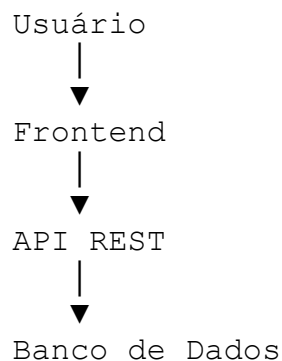
- renderização de componentes;
- interação do usuário;
- navegação entre páginas;
- formulários;
- estados de carregamento;
- mensagens de erro;
- integração com a API.

Os testes devem simular o comportamento real do usuário sempre que possível.

8.8 Testes End-to-End (E2E)

Os testes E2E validam o funcionamento completo da aplicação.

Fluxo simplificado:



Exemplos de cenários:

- login;
- cadastro de usuários;
- gerenciamento de publicadores;
- cadastro de territórios;
- emissão de relatórios;
- logout.

Esses testes garantem que os principais fluxos de negócio permaneçam operacionais.

8.9 Testes Não Funcionais

Além dos testes funcionais, deverão ser realizados testes relacionados à qualidade do sistema.

Categoria	Objetivo
Desempenho	Avaliar tempos de resposta
Carga	Verificar comportamento sob volume elevado
Estresse	Identificar limites da aplicação
Segurança	Validar autenticação, autorização e proteção contra vulnerabilidades
Compatibilidade	Confirmar funcionamento em diferentes navegadores e dispositivos

Os testes não funcionais complementam a validação da solução em cenários reais de uso.

8.10 Ferramentas Utilizadas

As seguintes ferramentas serão utilizadas para apoiar a estratégia de testes.

Ferramenta	Finalidade
JUnit 5	Testes unitários do backend
Mockito	Simulação de dependências
Spring Boot Test	Testes de integração
Testcontainers	Banco de dados temporário para testes
Vitest	Testes unitários do frontend
React Testing Library	Testes de componentes React
Playwright	Testes End-to-End
Postman / Insomnia	Validação manual da API
GitHub Actions	Execução automática dos testes

A escolha dessas ferramentas considera integração com a stack tecnológica do GTP.

8.11 Cobertura de Código

A cobertura de testes deverá ser monitorada continuamente.

Metas recomendadas:

Camada	Cobertura Mínima
Services	$\geq 90\%$
Controllers	$\geq 80\%$
Repositories	$\geq 80\%$

Camada	Cobertura Mínima
Hooks React	$\geq 90\%$
Componentes críticos	$\geq 85\%$
Cobertura geral	$\geq 80\%$

A cobertura é um indicador importante, mas não substitui testes bem planejados.

8.12 Critérios para Aprovação

Uma funcionalidade somente poderá ser integrada quando:

- os testes automatizados forem executados com sucesso;
- não existirem erros críticos;
- a revisão de código for aprovada;
- a documentação estiver atualizada;
- a cobertura mínima for mantida;
- a pipeline de CI for concluída com sucesso.

Esses critérios ajudam a manter a estabilidade da aplicação.

8.13 Qualidade do Código

Além dos testes, a qualidade será avaliada considerando:

- legibilidade;
- modularização;
- reutilização;
- conformidade com Clean Code;
- aplicação dos princípios SOLID;
- ausência de duplicação;
- complexidade controlada.

A qualidade do código é parte integrante do processo de desenvolvimento.

8.14 Boas Práticas

Durante a implementação de testes recomenda-se:

- escrever testes simples e objetivos;
- nomear os testes de forma descritiva;
- manter independência entre cenários;
- evitar dependências externas desnecessárias;

- utilizar dados previsíveis;
- atualizar os testes sempre que o comportamento mudar;
- remover testes obsoletos.

8.15 Antipadrões

Devem ser evitados:

- testes dependentes entre si;
- testes frágeis;
- excesso de *mocks* sem necessidade;
- duplicação de cenários;
- testes lentos para funcionalidades simples;
- ignorar testes falhos;
- reduzir cobertura para acelerar entregas.

Esses antipadrões diminuem a confiabilidade da suíte de testes.

8.16 Checklist de Qualidade

Antes de concluir uma funcionalidade, verificar:

- testes unitários executados;
- testes de integração aprovados;
- testes da API executados;
- testes do frontend atualizados;
- cobertura mantida;
- documentação revisada;
- análise de código concluída;
- pipeline CI aprovada.

8.17 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a estrutura das classes testadas.
Documento 07 – Arquitetura Frontend	Complementa os testes da interface e componentes.
Documento 10 – API REST	Especifica os endpoints validados nos testes da API.

Documento	Relação
Documento 13 – Estratégia de Testes	Detalha a arquitetura completa da estratégia de testes adotada.
Documento 14 – Deploy e Infraestrutura	Define a execução automatizada dos testes na pipeline de CI/CD.
Documento 15 – Manual do Desenvolvedor	Consolida as práticas de qualidade para o desenvolvimento diário.

8.18 Conclusão do Capítulo

A adoção de uma estratégia consistente de testes e qualidade é essencial para garantir a confiabilidade e a evolução sustentável do GTP. A combinação de testes unitários, de integração, de API, de interface e End-to-End, aliada à integração contínua e às revisões de código, reduz significativamente a ocorrência de regressões e aumenta a confiança nas entregas.

Seguir as diretrizes apresentadas neste capítulo permite que a equipe desenvolva novas funcionalidades com maior segurança, preservando a estabilidade do sistema e assegurando um elevado padrão de qualidade ao longo de todo o ciclo de vida do projeto.

9 – Contribuição para o Projeto

9.1 Objetivo

Este capítulo estabelece as diretrizes para contribuição no desenvolvimento do **Gestor de Territórios e Publicações (GTP)**, definindo o processo de colaboração, responsabilidades, fluxo de trabalho e critérios de qualidade que deverão ser seguidos por todos os participantes do projeto.

O objetivo é garantir que a evolução do sistema ocorra de forma organizada, rastreável, colaborativa e alinhada aos padrões técnicos definidos na documentação.

9.2 Princípios de Colaboração

Toda contribuição deverá observar os seguintes princípios:

- respeito aos padrões arquiteturais;
- comunicação clara;

- colaboração entre os desenvolvedores;
- transparência nas alterações;
- documentação das decisões técnicas;
- foco na qualidade do código;
- melhoria contínua.

O conhecimento produzido durante o desenvolvimento deverá permanecer registrado no repositório e na documentação do projeto.

9.3 Processo de Onboarding

Novos colaboradores deverão seguir um processo de integração antes de iniciar contribuições.

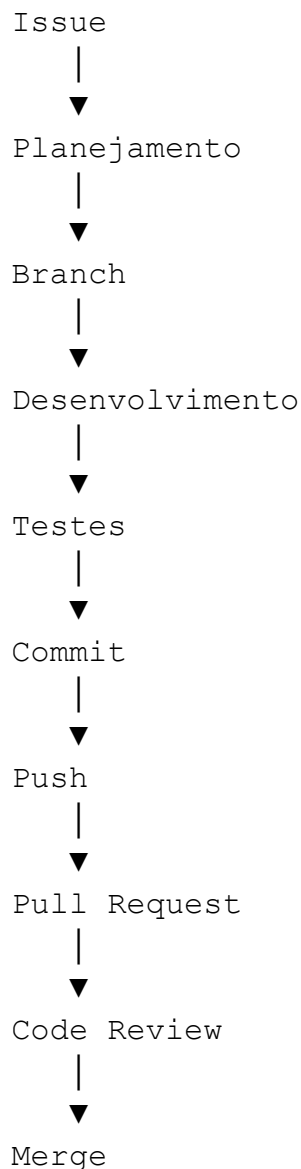
Etapas recomendadas:

1. Clonar o repositório oficial.
2. Configurar o ambiente de desenvolvimento.
3. Executar a aplicação localmente.
4. Ler a documentação principal do projeto.
5. Compreender a arquitetura do sistema.
6. Revisar os padrões de desenvolvimento.
7. Executar os testes automatizados.
8. Realizar uma pequena contribuição inicial.

Esse processo reduz o tempo de adaptação e promove alinhamento com as práticas do projeto.

9.4 Processo de Contribuição

Toda nova funcionalidade, melhoria ou correção deverá seguir o fluxo oficial.



Esse fluxo garante rastreabilidade e controle sobre todas as alterações realizadas.

9.5 Gestão de Issues

As **Issues** representam a unidade oficial de trabalho do projeto.

Cada issue deverá conter:

- título objetivo;
- descrição detalhada;
- contexto do problema;
- critérios de aceite;
- prioridade;

- responsável;
- etiquetas (*labels*);
- marco (*milestone*), quando aplicável.

Sempre que possível, uma contribuição deverá estar vinculada a uma issue correspondente.

9.6 Planejamento das Tarefas

As atividades poderão ser classificadas conforme sua natureza.

Categoria	Descrição
------------------	------------------

Funcionalidade	Implementação de novos recursos
----------------	---------------------------------

Correção	Ajuste de defeitos
----------	--------------------

Refatoração	Melhoria estrutural sem alteração funcional
-------------	---

Documentação	Atualização da documentação técnica
--------------	-------------------------------------

Testes	Implementação ou atualização de testes
--------	--

Infraestrutura	Configurações de deploy, Docker e CI/CD
----------------	---

Essa classificação facilita o acompanhamento da evolução do projeto.

9.7 Desenvolvimento da Solução

Durante a implementação de uma contribuição, recomenda-se:

- criar uma branch específica;
- manter commits pequenos e frequentes;
- seguir os padrões definidos no Manual do Desenvolvedor;
- implementar testes para a funcionalidade;
- atualizar a documentação quando necessário.

Alterações extensas devem ser divididas em partes menores sempre que possível.

9.8 Pull Request

Após concluir a implementação, deverá ser criado um Pull Request contendo:

- resumo da alteração;
- objetivo da implementação;
- referência à issue relacionada;
- evidências dos testes realizados;
- observações relevantes para os revisores.

O Pull Request deverá tratar apenas de um único objetivo, evitando misturar funcionalidades distintas.

9.9 Revisão de Código (Code Review)

Todo Pull Request deverá passar por revisão antes da integração.

Durante a revisão serão avaliados aspectos como:

- aderência aos padrões de desenvolvimento;
- qualidade do código;
- arquitetura;
- legibilidade;
- segurança;
- desempenho;
- cobertura de testes;
- documentação.

O objetivo da revisão é promover aprendizado e manter a qualidade do projeto.

9.10 Critérios para Aprovação

Uma contribuição poderá ser aprovada quando:

- atender aos requisitos definidos na issue;
- respeitar os padrões arquiteturais;

- possuir testes adequados;
- não introduzir regressões;
- manter a documentação atualizada;
- obter aprovação na revisão de código;
- concluir a pipeline de CI com sucesso.

Esses critérios asseguram a consistência das integrações.

9.11 Atualização da Documentação

Sempre que uma alteração modificar o comportamento da aplicação, deverá ser avaliada a necessidade de atualizar:

- documentação funcional;
- documentação técnica;
- diagramas;
- API REST;
- modelos de dados;
- manuais do desenvolvedor e do usuário.

A documentação deve evoluir juntamente com o código.

9.12 Comunicação da Equipe

A comunicação entre os colaboradores deverá ser:

- objetiva;
- respeitosa;
- técnica;
- documentada sempre que possível.

Decisões importantes deverão ser registradas em issues, Pull Requests ou documentos do projeto para facilitar consultas futuras.

9.13 Boas Práticas

Durante as contribuições recomenda-se:

- compreender o problema antes de implementar a solução;
- reutilizar componentes existentes;
- evitar duplicação de código;
- realizar testes antes do envio;
- manter Pull Requests pequenos;
- revisar o próprio código antes da submissão;
- responder aos comentários da revisão de forma colaborativa.

9.14 Antipadrões

Devem ser evitados:

- alterações diretamente na branch principal;
- Pull Requests excessivamente grandes;
- commits sem descrição adequada;
- mistura de funcionalidades distintas em uma mesma contribuição;
- código sem testes;
- documentação desatualizada;
- resolução de conflitos diretamente na interface do GitHub sem análise prévia.

Esses antipadrões dificultam a revisão e aumentam o risco de erros.

9.15 Checklist para Contribuição

Antes de solicitar a revisão de uma contribuição, verificar:

- issue relacionada atualizada;
- branch sincronizada;
- código compilando corretamente;

- testes executados;
- documentação revisada;
- padrões de desenvolvimento respeitados;
- commits organizados;
- Pull Request preenchido corretamente.

9.16 Ética e Conduta

Todos os colaboradores deverão manter um ambiente de trabalho colaborativo, respeitoso e profissional.

Espera-se que cada participante:

- trate os demais com cordialidade;
- aceite críticas construtivas;
- forneça feedback técnico fundamentado;
- respeite diferentes opiniões;
- contribua para um ambiente inclusivo e cooperativo.

A colaboração saudável é essencial para a evolução do projeto.

9.17 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura Geral	Define os princípios arquiteturais seguidos durante as contribuições.
Documento 06 – Arquitetura Backend	Orienta a implementação de novas funcionalidades no backend.
Documento 07 – Arquitetura Frontend	Define os padrões para evolução da interface.
Documento 13 – Estratégia de Testes	Estabelece os testes obrigatórios antes da integração.

Documento	Relação
Documento 14 – Deploy e Infraestrutura	Define o fluxo de CI/CD utilizado na validação das contribuições.
Documento 15 – Manual do Desenvolvedor	Consolida todas as diretrizes para desenvolvimento colaborativo.

9.18 Conclusão do Capítulo

Um processo de contribuição bem definido é fundamental para garantir a evolução organizada do GTP. A utilização de Issues, branches específicas, Pull Requests, revisões de código e integração contínua permite que novas funcionalidades sejam incorporadas com segurança, qualidade e rastreabilidade.

Ao seguir as diretrizes apresentadas neste capítulo, os colaboradores contribuem para um ambiente de desenvolvimento colaborativo, sustentável e alinhado às melhores práticas da engenharia de software.

10 – Conclusão e Próximos Passos

10.1 Objetivo

Este capítulo encerra o **Manual do Desenvolvedor** do **Gestor de Territórios e Publicações (GTP)**, consolidando os princípios, padrões e processos apresentados ao longo do documento.

Seu propósito é reforçar a importância da padronização do desenvolvimento, destacar o papel da documentação técnica na evolução do sistema e estabelecer as diretrizes para as próximas etapas do projeto.

10.2 Síntese do Manual

Ao longo deste documento foram definidos os principais aspectos relacionados ao desenvolvimento do GTP, incluindo:

- preparação do ambiente de desenvolvimento;
- estrutura do projeto;

- padrões de desenvolvimento;
- convenções de codificação;
- fluxo de trabalho com Git e GitHub;
- execução da aplicação;
- depuração e solução de problemas;
- estratégia de testes e qualidade;
- processo de contribuição para o projeto.

Essas diretrizes formam uma base consistente para garantir que todas as implementações mantenham o mesmo padrão de qualidade, organização e legibilidade.

10.3 Importância da Padronização

A adoção de padrões comuns proporciona diversos benefícios ao projeto:

- maior facilidade de manutenção;
- redução do acoplamento entre componentes;
- aumento da produtividade da equipe;
- simplificação da revisão de código;
- menor incidência de defeitos;
- maior previsibilidade durante a evolução do sistema.

A padronização também reduz o tempo necessário para integração de novos colaboradores e facilita a continuidade do desenvolvimento.

10.4 Evolução Contínua

O GTP é um projeto em constante evolução. Dessa forma, este manual deverá ser atualizado sempre que ocorrerem mudanças significativas na arquitetura, nas tecnologias utilizadas ou nos processos de desenvolvimento.

Entre as situações que podem exigir atualização da documentação estão:

- adoção de novas bibliotecas ou frameworks;
- alterações na arquitetura do sistema;

- mudanças no fluxo de trabalho;
- revisão dos padrões de desenvolvimento;
- inclusão de novos módulos;
- evolução da infraestrutura;
- atualização das ferramentas de testes ou integração contínua.

A documentação deverá acompanhar o ritmo de evolução do código-fonte.

10.5 Responsabilidade dos Desenvolvedores

Cada desenvolvedor é responsável por:

- compreender a arquitetura do sistema;
- seguir os padrões estabelecidos;
- produzir código limpo e bem documentado;
- manter a qualidade das implementações;
- escrever e atualizar testes;
- revisar cuidadosamente suas alterações antes da integração;
- contribuir para a melhoria contínua da documentação.

O cumprimento dessas responsabilidades fortalece a qualidade do projeto e reduz riscos durante sua evolução.

10.6 Boas Práticas Reforçadas

Ao longo do desenvolvimento do GTP, recomenda-se manter como referência permanente os seguintes princípios:

- escrever código simples e legível;
- aplicar os princípios SOLID;
- seguir as práticas de Clean Code;
- reutilizar componentes e serviços;
- manter a documentação sincronizada com o código;
- realizar testes antes de cada integração;

- registrar decisões técnicas relevantes;
- promover revisões de código construtivas.

Essas práticas devem orientar todas as contribuições ao projeto.

10.7 Próximas Etapas do Projeto

Com a conclusão desta documentação, o projeto encontra-se preparado para avançar para as fases de implementação e evolução contínua.

As próximas etapas previstas incluem:

1. implementação completa do backend em Spring Boot;
2. desenvolvimento da interface web em React;
3. construção e validação da API REST;
4. implementação das regras de negócio;
5. configuração da infraestrutura Docker;
6. automação das pipelines de CI/CD;
7. ampliação da cobertura de testes;
8. implantação dos ambientes de homologação e produção;
9. monitoramento contínuo da aplicação;
10. evolução funcional conforme novas necessidades identificadas.

Essas atividades serão executadas em conformidade com a arquitetura e os padrões definidos na documentação.

10.8 Visão de Longo Prazo

O GTP foi concebido para ser uma plataforma moderna, modular e escalável, capaz de evoluir de forma sustentável.

A arquitetura adotada permite:

- crescimento gradual dos módulos;
- integração com novos serviços;
- ampliação da infraestrutura;

- suporte a novas funcionalidades;
- adaptação a futuras tecnologias.

Essa visão garante que o sistema permaneça preparado para atender novas demandas sem comprometer sua estabilidade.

10.9 Documentação como Parte do Produto

A documentação técnica não deve ser tratada como um artefato secundário, mas como parte integrante do software.

Ela desempenha papel essencial na:

- preservação do conhecimento do projeto;
- comunicação entre desenvolvedores;
- redução da curva de aprendizado;
- padronização das implementações;
- manutenção de longo prazo.

Por esse motivo, toda alteração relevante no sistema deverá ser refletida na documentação correspondente.

10.10 Considerações Finais

O **Manual do Desenvolvedor** consolida as diretrizes necessárias para garantir que o desenvolvimento do **Gestor de Territórios e Publicações (GTP)** ocorra de forma organizada, segura e alinhada às melhores práticas da engenharia de software.

A combinação entre arquitetura bem definida, padrões de codificação, processos de colaboração, testes automatizados e documentação estruturada estabelece uma base sólida para a construção de uma aplicação robusta, escalável e de fácil manutenção.

À medida que o projeto evoluir, este manual deverá permanecer como referência oficial para todos os desenvolvedores, assegurando consistência nas implementações e facilitando a incorporação de novas funcionalidades e colaboradores.

Encerramento do Documento

Com este capítulo, conclui-se o **Documento 15 – Manual do Desenvolvedor**, que reúne as orientações para configuração do ambiente, organização do código, padrões de desenvolvimento, fluxo de trabalho, testes, depuração e colaboração.

Este documento complementa os demais artefatos de arquitetura, banco de dados, API, infraestrutura e testes, formando um conjunto integrado de documentação técnica para o **Gestor de Territórios e Publicações (GTP)**.